

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03/08/2026

1 Objective

Classify emails as spam or ham with Logistic Regression and SVM, compare four SVM kernels (linear, polynomial, RBF, sigmoid), tune both with GridSearchCV, and see where a straight line stops being enough.

2 Problem Statement

Given the same Spambase dataset as Experiment 2, 57 numeric features per email plus a spam/ham label, classify each email with a linear probabilistic model and a margin based model, and check whether a curved kernel boundary actually helps here.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle, somesh24/spambase
Number of Samples	4601
Number of Features	57
Number of Classes	2 (spam = 1, ham = 0)
Missing Values	none
Train-Test Split	80/20, stratified, with 5 fold cross validation

4 Exploratory Data Analysis

- Class distribution: 2788 ham vs 1813 spam.
- Correlation heatmap across all 57 features.
- Right skewed word frequency distributions, same pattern as Experiment 2.

¹GitHub Repository: <https://github.com/sanjay-6727/Introduction-to-Machine-Learning->

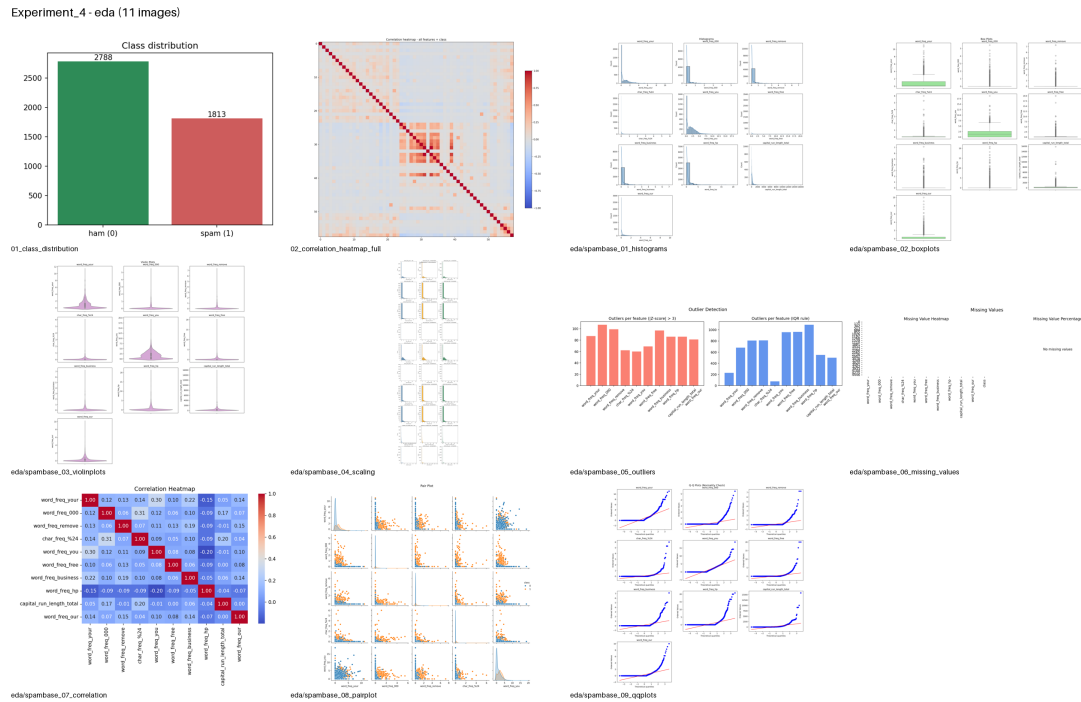


Figure 1: Class distribution, full correlation heatmap, and histogram, box plot, violin, scaling, outlier, pair plot and Q-Q panels for the ten most predictive features.

Inference:

Same underlying dataset as Experiment 2, so the same features (word_freq_your, word_freq_000, exclamation and dollar sign frequencies) stand out as the strongest spam indicators. The pair plot shows the spam and ham clusters overlapping rather than sitting in clearly separated regions, an early hint that a linear boundary might already do most of the work.

5 Data Preprocessing

- No missing values, all features numeric, no encoding needed.
- StandardScaler applied before both Logistic Regression and SVM.
- 80/20 stratified split, evaluated further with 5 fold cross validation.

Listing 1: Grid search for Logistic Regression and SVM across four kernels

```

1 lr_grid = GridSearchCV(LogisticRegression(max_iter=5000),
2                         lr_param_grid, cv=5, scoring="accuracy")
3 lr_grid.fit(X_train_scaled, y_train)
4
5 svm_param_grid = [
6     {"kernel": ["linear"], "C": [0.1, 1, 10]},
7     {"kernel": ["rbf"], "C": [0.1, 1, 10], "gamma": ["scale", "
      auto"]},
8     {"kernel": ["poly"], "C": [0.1, 1, 10], "degree": [2, 3]},
9 ]

```

```

10 svm_grid = GridSearchCV(SVC(), svm_param_grid, cv=5, scoring="
    accuracy")
11 svm_grid.fit(X_train_scaled, y_train)

```

6 Machine Learning Algorithm

Logistic Regression models $P(y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$, trained by minimizing log loss. C controls regularization strength, small C means a simpler boundary.

Support Vector Machine finds the hyperplane that maximizes the margin between classes, using only the closest support vectors. The kernel trick lets it draw non-linear boundaries, linear stays flat, RBF and polynomial curve, sigmoid behaves like a single layer neural network. C trades margin width against misclassification, γ controls how far a point's influence reaches.

7 Experimental Setup

Python Version	3.11.9
Scikit-Learn Version	1.7.2
IDE	Anaconda Navigator / VS Code
Operating System	Windows 11
Hardware	

8 Hyperparameter Selection

Parameter	Value
LR grid	penalty L1/L2, C in {0.01,0.1,1,10,100}, solver liblinear/saga
SVM grid	kernel in {linear,poly,rbf,sgmold}, C in {0.1,1,10}, gamma, degree(poly)

Model		Search Method	Best Parameters	Best CV Accuracy
Logistic Regression		Grid Search	C=1, penalty=L2, solver=saga	0.9242
SVM		Grid Search	C=10, gamma=scale, kernel=rbf	0.9332

9 Results

Logistic Regression Metric	Value
Accuracy	0.9305
Precision	0.9211
Recall	0.9008
F1 Score	0.9109

SVM Kernel-wise Performance

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9273	0.9063	11.98
RBF	0.9207	0.8976	1.52
Sigmoid	0.8893	0.8534	2.14
Poly	0.9164	0.8902	1.68

Comparative Analysis

Criterion	Logistic Regression	SVM
Accuracy	0.9305	0.9273
Model Complexity	Low	High
Training Time	Low	High
Interpretability	High	Low

10 Result Visualization

Experiment_4 - results (7 Images)

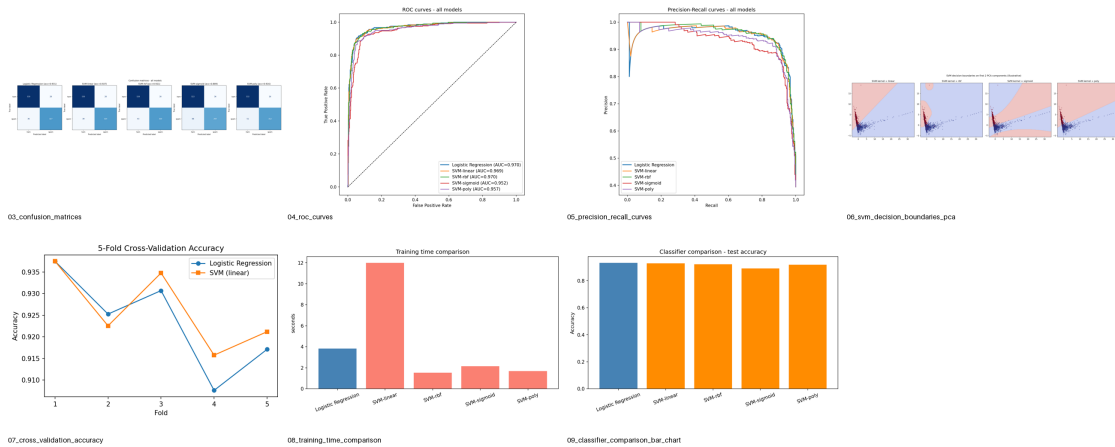


Figure 2: Confusion matrices and ROC/precision-recall curves for Logistic Regression and all four SVM kernels, SVM decision boundaries on the first two PCA components, cross validation accuracy, and training time and accuracy comparison bars.

Inference:

Logistic Regression and linear SVM confusion matrices are almost identical, a strong sign the true boundary here really is close to linear. The PCA decision boundary plot backs this up, linear and RBF carve the space up similarly while sigmoid looks noticeably more erratic, matching sigmoid's worst accuracy (0.8893). Training time is the most lopsided chart, linear SVM takes about 12 seconds against 1.5 to 2 seconds for the other kernels, likely because Spambase is not perfectly linearly separable, forcing the solver to work harder for a margin with few mistakes.

11 Discussion

Logistic Regression wins on nearly every axis, marginally higher test accuracy (0.9305 vs SVM's best 0.9273), far faster training, and coefficients that can actually be inspected. RBF edges out Logistic Regression slightly on CV accuracy (0.9332 vs 0.9242), so the two are close, but that edge does not seem worth the extra cost and lost interpretability for spam filtering, where seeing which features drove a flag matters. Sigmoid's weak showing matches its reputation of only working in a narrow parameter range.

12 Conclusion

For Spambase, Logistic Regression matches or slightly beats every SVM kernel while training much faster and staying interpretable, making it the practical choice here. RBF SVM stays a reasonable second option if a larger or higher dimensional dataset introduced more real non-linear structure.

13 Learning Outcomes

- Confirmed that a kernel method is not automatically better, on data that is already close to linearly separable, the extra flexibility barely helps and costs real training time.
- Practiced reading a PCA projected decision boundary as a sanity check on why one kernel is outperforming another.
- Saw concretely how C and γ change an SVM boundary, not just as a definition but in the actual accuracy numbers.
- Learned to weigh accuracy against interpretability and training cost when picking a model, not just chase the highest number.

14 References

1. Stephen Marsland, "Machine Learning - An Algorithmic Perspective", 2nd Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2015.
2. Ethem Alpaydin, "Introduction to Machine Learning", 3rd Edition, The MIT Press, 2014.